

AULA 11 • PROGRAMAÇÃO ORIENTADA A OBJETOS

Tratamento de Exceções

`try` · `catch` · `finally` · `throw` · exceções customizadas — como manter o programa **de pé** quando algo dá errado.

Java • P00 • Prof. Rob Silva

O caminho de hoje.

- 01 Por que tratar erros
- 02 A história do financiamento
- 03 `throw` : a classe que se defende
- 04 `try` / `catch` / `finally`
- 05 Exceções customizadas
- 06 De volta ao estoque MegaStore
- 07 Exercícios

Quando o programa quebra.

Até agora, se algo dá errado — uma conta impossível, um dado inválido — o Java para tudo e joga um erro feio na tela. Hoje a gente assume o controle: prever o erro, tratar com elegância e manter o programa de pé.

```
// o que o Java cospe na tela  
Exception in thread "main"  
java.lang.ArithmeticException: / by zero
```

O CENÁRIO

Fomos contratados por um banco.

A missão: calcular o valor da parcela de um financiamento. A conta é simples — tira a entrada do valor total e divide pelo número de parcelas.

```
// classe Financiamento
public double prestacao() {
    return (valorTotal - entrada) / parcelas;
}
```



Encher o Main de if.

O banco exige entrada de no mínimo 20% e no mínimo 6 parcelas. O caminho fácil? Espalhar ifs no programa principal antes de criar o objeto.

```
// classe Principal (Main) – regras espalhadas
if (entrada < valorTotal * 0.2) {
    System.out.println("Erro: entrada mínima de 20%.");
} else if (parcelas < 6) {
    System.out.println("Erro: mínimo de 6 parcelas.");
} else {
    Financiamento f =
        new Financiamento(valorTotal, entrada, parcelas);
    System.out.println("Parcela: " + f.prestacao());
}
```

Funciona... mas é gambiarra. A regra do banco ficou grudada no programa principal.

Regra de negócio no lugar errado.

- A regra do banco ficou grudada na tela
(`System.out.println`).
- Virou app ou site?
Reescrever tudo de novo.
- A classe `Financiamento` não sabe se proteger sozinha.

Quem conhece as regras do financiamento é o próprio **Financiamento**.

🛡️ A classe se defende.

Movemos as regras para dentro do construtor. Se os dados violam a regra, o objeto nem nasce — ele dispara um `throw`.

```
// construtor da classe Financiamento
public Financiamento(double valorTotal,
                      double entrada, int parcelas) {
    if (entrada < valorTotal * 0.2) {
        throw new RuntimeException(
            "A entrada deve ser ao menos 20% do total.");
    }
    if (parcelas < 6) {
        throw new RuntimeException(
            "O número mínimo de parcelas é 6.");
    }
    this.valorTotal = valorTotal;
    this.entrada = entrada;
    this.parcelas = parcelas;
}
```

O Main volta a respirar.

O programa principal só tenta (`try`) criar o financiamento. Se a regra for violada, o `catch` pega a exceção e mostra a mensagem — sem quebrar.

```
// classe Principal (Main) – limpo
try {
    Financiamento f =
        new Financiamento(1000.0, 100.0, 10); // entrada 10%
    System.out.println("Prestação: " + f.prestacao());
}
catch (RuntimeException e) {
    System.out.println(e.getMessage());
}
```

A entrada de 10% viola a regra, então o construtor lança — e o `catch` imprime a mensagem sem derrubar o programa.

A GRANDE IDEIA

O programa não morre. Ele se recupera.

É isso que uma `Exception` faz por você.

try • catch • finally.

try

O código que **pode** dar errado. Você tenta executar aqui dentro.

catch

O plano B quando dá errado. Recebe a exceção e decide o que fazer.

finally

Roda **sempre**, com erro ou sem — fechar arquivo, liberar recurso.

Os três blocos juntos.

```
try {
    Financiamento f =
        new Financiamento(1000.0, 100.0, 10);
    System.out.println("Prestação: " + f.prestacao());
}
catch (RuntimeException e) {
    System.out.println("Falhou: " + e.getMessage());
}
finally {
    // roda SEMPRE – com erro ou sem
    System.out.println("Cálculo finalizado.");
}
```

Deu certo ou deu erro, o `finally` imprime "**Cálculo finalizado.**" antes de seguir em frente.

NÃO CONFUNDA

throw × throws.

throw

Dispara a exceção **agora**, no meio do código.

```
// dispara a exceção AGORA  
throw new RuntimeException("Valor inválido");
```

throws

Avisa na **assinatura** que o método pode lançar.

```
// avisa que o método pode lançar  
public void salvar() throws IOException {  
    // ...  
}
```

Crie a sua própria exceção.

`RuntimeException` genérica resolve, mas uma exceção com nome próprio conta a história. `FinanciamentoInvalidoException` já diz o que aconteceu — e você pode tratar cada erro do seu jeito.

Uma classe de exceção.

```
// a nossa própria exceção
public class FinanciamentoInvalidoException
    extends RuntimeException {

    public FinanciamentoInvalidoException(String msg) {
        super(msg);
    }
}
```

Herdar de `RuntimeException` já traz todo o comportamento pronto. O construtor só repassa a mensagem com `super(msg)`.

Usando a exceção com nome.

```
// no construtor: lança com nome próprio  
if (entrada < valorTotal * 0.2) {  
    throw new FinanciamentoInvalidoException(  
        "Entrada menor que 20%.");  
}
```

```
// no Main: captura o tipo específico  
catch (FinanciamentoInvalidoException e) {  
    System.out.println("Regra do banco: "  
        + e.getMessage());  
}
```

O Produto também se defende.

O mesmo padrão vale para o estoque da MegaStore: o construtor de `Produto` recusa preço ou quantidade negativos. O menu tenta cadastrar dentro de um try-catch e continua vivo.

```
// classe Produto do estoque (JAVA-104)
public Produto(String nome, double preco,
               int quantidade, String categoria) {
    if (preco < 0) {
        throw new RuntimeException(
            "Preço não pode ser negativo.");
    }
    if (quantidade < 0) {
        throw new RuntimeException(
            "Quantidade não pode ser negativa.");
    }
    this.nome = nome;
    this.preco = preco;
    this.quantidade = quantidade;
    this.categoria = categoria;
}
```


As 3 lições da aula.

1 • Onde fica a regra

Regra de negócio **não fica no Main**. Ela pertence à classe que a conhece.

2 • Autovalidação

Toda classe deve se **autovalidar** — nunca existir num estado inválido.

3 • Código limpo

`try` / `catch` / `finally` substitui if-else gigante e deixa o código profissional.

PAY-42

MÉDIO

TO DO

BACKEND

PAGFÁCIL · FINTECH

A conta que não deixa você errar

HISTÓRIA DO USUÁRIO

Como cliente do PagFácil, quero que o app me impeça de sacar mais do que tenho na conta, para eu nunca ficar com saldo negativo sem querer.

CRITÉRIOS DE ACEITE

- ☒ No construtor de `ContaBancaria`, lançar `throw new RuntimeException(...)` se o saldo inicial for negativo.
- ☒ No método `sacar(double valor)`, lançar exceção se `valor <= 0` ou se `valor > saldo` ("**Saldo insuficiente**").
- ☒ No `Main`, envolver o saque em `try / catch` e mostrar a mensagem — **sem quebrar** o programa.

#pagfacil

#conta

#excecoes

Restrição · A conta se valida sozinha – sem if de validação no Main.

EVT-17

MÉDIO

TO DO

BACKEND

TICKETZ • EVENTOS

Um erro com nome: ingresso esgotado

HISTÓRIA DO USUÁRIO

Como organizador de eventos na TicketZ, quero um erro específico quando alguém tenta comprar ingresso de um evento esgotado, para deixar claro o motivo da recusa.

CRITÉRIOS DE ACEITE

- ☒ Criar `IngressoEsgotadoException` `extends RuntimeException` com construtor que recebe a mensagem (`super(msg)`).
- ☒ Na classe `Evento`, no método `venderIngresso()`, lançar `throw new IngressoEsgotadoException(...)` quando não houver ingressos.
- ☒ No `Main`, capturar `IngressoEsgotadoException` num `catch` específico e avisar o cliente.

#ticketz

#evento

#excecao-customizada

Restrição • A nova classe deve estender `RuntimeException` e reaproveitar `super(msg)`.

CAD-88

DESAFIO

TO DO

APP

CADSHOW • CADASTRO

O formulário à prova de dedo errado

HISTÓRIA DO USUÁRIO

Como usuário do CadShow, quero que o cadastro não quebre quando eu digitar uma letra no campo de idade, para conseguir concluir o meu cadastro.

CRITÉRIOS DE ACEITE

- ☒ Ler a idade com `Scanner` e converter com `Integer.parseInt(...)` dentro de um `try`.
- ☒ Tratar `NumberFormatException` num `catch` ("**Digite apenas números.**") e usar `finally` que sempre exibe "**Cadastro finalizado.**".
- ☒ Digitar texto onde se espera número **nunca pode encerrar** o programa com erro.

#cadshow

#formulario

#try-catch-finally

Desafio extra • Repetir a leitura até vir uma idade válida (laço + try/catch) e rejeitar idade negativa com um `throw`.

